BUSINESS ASSISTANT

A Multi-Agent Conversational Analytics System

A Final Year Project Report
Submitted in Partial Fulfillment of the Requirements
for the Degree of Bachelor of Science in Information Technology

Submitted By:
Muhammad Ammar Zamad Khan
Student ID: [Student ID]

Supervisor:
[Supervisor Name]

Department of Information Technology
[University Name]

May 2026

ABSTRACT

This report presents the design, development, and evaluation of the Business Assistant — a multi-agent conversational analytics system that enables non-technical users to query relational databases using plain English, review automatically generated SQL queries before execution, and visualise results as interactive charts. The system addresses three fundamental limitations of existing natural language to SQL solutions: the inability to handle large schemas accurately, the lack of domain-specific terminology awareness, and the absence of human oversight over machine-generated database queries.

The architecture employs four cooperative agents — a Supervisor, a Schema Agent, a SQL Agent, and an Analysis Agent — orchestrated by LangGraph, a stateful agent framework built on LangChain. Schema linking is performed using ChromaDB vector embeddings with the all-MiniLM-L6-v2 sentence transformer model, which retrieves only the tables relevant to a given question rather than passing the entire database schema to the language model. A user-maintained business glossary stores domain-specific term definitions that are automatically injected into SQL generation prompts. Human-in-the-loop (HITL) SQL approval leverages LangGraph's interrupt() primitive to pause the agent pipeline mid-execution, present the draft SQL to the user for review or editing, and resume only upon explicit approval.

The system was implemented using FastAPI for the backend, Next.js 14 with the App Router for the frontend, DuckDB for in-process analytical query execution, and PostgreSQL for operational data persistence. The project was completed as a fourteen-day solo build, producing 75 automated tests, an accuracy evaluation harness covering 20 ground-truth question-answer pairs, and a fully functional deployment-ready application.

Evaluation results demonstrate that the system correctly handles multi-table JOIN queries, aggregation queries, and domain-specific terminology resolution, with a measured accuracy rate of approximately 85% on the evaluation harness, rising to 90% after one iteration. The report discusses architectural trade-offs, the specific challenges of implementing stateful HITL workflows in a web application, and directions for future extension.

Keywords: multi-agent systems, natural language to SQL, human-in-the-loop, LangGraph, conversational analytics, schema linking, business intelligence.

TABLE OF CONTENTS

1. Introduction

1.1 Background and Motivation

1.2 Problem Statement

1.3 Objectives

1.4 Scope and Limitations

1.5 Report Organisation

2. Literature Review

2.1 Natural Language Interfaces to Databases

2.2 Multi-Agent Systems

2.3 Human-in-the-Loop Machine Learning

2.4 Schema Linking and Embedding-Based Retrieval

2.5 Summary and Research Gap

3. System Design and Architecture

3.1 High-Level Architecture

3.2 Agent Design

3.3 State Management

3.4 Data Layer Design

3.5 Authentication and Security

4. Implementation

4.1 Technology Stack

4.2 Backend Implementation

4.3 Agent Implementation

4.4 Frontend Implementation

4.5 Testing Strategy

5. Evaluation and Results

5.1 Accuracy Evaluation

5.2 System Performance

5.3 Usability Observations

6. Discussion

6.1 Meeting Objectives

6.2 Architectural Trade-offs

6.3 Limitations

7. Conclusion and Future Work

7.1 Conclusion

7.2 Future Work

References

Appendices

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The ability to extract insights from organisational data has become a critical competitive advantage in modern business environments. Traditionally, this capability has been restricted to individuals with technical expertise in structured query language (SQL), representing a significant barrier for business stakeholders who possess domain knowledge but lack programming skills. Analysts, managers, and executives who understand their business context intimately are often dependent on engineering teams to translate their information needs into database queries, introducing delays and communication overhead that reduce organisational agility.

Recent advances in large language models (LLMs) have created an opportunity to bridge this gap through natural language interfaces to databases. However, deploying such systems in production business environments introduces new challenges: schemas in real organisations contain hundreds of tables with overlapping names; LLMs lack awareness of company-specific terminology and business rules; and fully autonomous SQL execution presents correctness and security risks that are unacceptable in contexts where query results inform business decisions.

This project addresses these challenges through the design and implementation of the Business Assistant — a multi-agent conversational analytics system that combines LLM-powered SQL generation with semantic schema linking, domain knowledge injection, and mandatory human review before query execution.

1.2 Problem Statement

Existing natural language to SQL (NL2SQL) systems suffer from three specific limitations. First, naive implementations pass the entire database schema into the LLM prompt, causing accuracy to degrade as the schema grows. Second, generic LLMs have no knowledge of organisation-specific terminology — a query about 'revenue' will produce incorrect SQL unless the system knows the company-specific definition. Third, fully autonomous SQL execution provides no mechanism for users to catch incorrect queries before they execute, which is particularly dangerous in analytical workflows where a subtly wrong aggregation can lead to materially incorrect business decisions.

1.3 Objectives

This project has the following objectives:

- Design and implement a multi-agent architecture that decomposes the NL2SQL pipeline into specialist agents with single, testable responsibilities.

- Implement semantic schema linking using vector embeddings to retrieve only relevant tables rather than exposing the full schema to the language model.
- Provide a user-maintained business glossary that automatically injects domain-specific definitions into SQL generation prompts.
- Implement a human-in-the-loop SQL approval mechanism that pauses query execution, presents the draft SQL for review and optional editing, and resumes only upon explicit approval.
- Deliver a production-grade web application with JWT authentication, persistent conversation history, CSV data upload, and interactive chart visualisation.
- Achieve measurable accuracy on a ground-truth evaluation harness of 20 representative business questions.

1.4 Scope and Limitations

The project is scoped as a fourteen-day solo final year project targeting small-to-medium datasets held in DuckDB or uploaded as CSV files. Multi-tenant deployment, cloud data warehouse integration (e.g., AWS Athena), and Docker-based packaging are identified as post-project extensions. The accuracy evaluation is conducted against a single sample dataset; generalisation to arbitrary schemas is acknowledged as an area for future work.

1.5 Report Organisation

Chapter 2 reviews relevant literature on NL2SQL systems, multi-agent architectures, human-in-the-loop machine learning, and schema linking techniques. Chapter 3 presents the system design and architecture. Chapter 4 describes the implementation. Chapter 5 presents evaluation results. Chapter 6 discusses findings and limitations. Chapter 7 concludes the report and outlines future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Natural Language Interfaces to Databases

Research on natural language interfaces to databases (NLIDB) dates to the 1970s with early rule-based systems such as LUNAR (Woods, 1973) and LADDER (Hendrix et al., 1978). The field advanced significantly with deep learning approaches, particularly sequence-to-sequence models trained on annotated NL-SQL pairs. The WikiSQL benchmark (Zhong et al., 2017) and the more complex Spider benchmark (Yu et al., 2018) established standardised evaluation frameworks and spurred competitive model development.

The advent of large language models introduced a paradigm shift. DAIL-SQL (Gao et al., 2023) and DIN-SQL (Pourreza and Rafiei, 2023) demonstrated that GPT-4 with carefully engineered prompts could achieve state-of-the-art performance on Spider without task-specific fine-tuning. BIRD (Li et al., 2023) extended evaluation to real-world databases with dirty values and complex business rules, highlighting that benchmark performance does not automatically transfer to production deployment. These findings motivate the design choices in the Business Assistant, particularly the need for schema linking and glossary injection.

2.2 Multi-Agent Systems

Multi-agent systems for LLM applications have received increasing attention since the publication of AutoGPT (Significant Gravitas, 2023) and BabyAGI (Nakajima, 2023). The Supervisor with Specialists pattern, recommended by LangChain and adopted in this project, follows a hierarchical decomposition where a coordinator agent routes tasks to specialist sub-agents. This pattern provides modularity, testability, and the ability to evolve individual specialist capabilities without affecting others (Chase, 2023).

LangGraph (Anthropic/LangChain, 2024) extends this model with first-class support for stateful, cyclic agent workflows. Unlike linear LangChain chains, LangGraph represents the agent pipeline as a directed graph with typed shared state, conditional routing, and — critically for this project — an interrupt() primitive that suspends graph execution and serialises state to a persistent checkpointer. These properties make LangGraph uniquely suited to human-in-the-loop workflows.

2.3 Human-in-the-Loop Machine Learning

Human-in-the-loop (HITL) machine learning refers to system designs that incorporate human judgement at specific points in an otherwise automated pipeline. In the context of NL2SQL, HITL approval serves as a correctness gate: the user reviews the generated

query before it executes, catching semantic errors that automated validation cannot detect. Amershi et al. (2014) identified interactive machine learning as a paradigm where human feedback is collected actively to improve system behaviour. In production analytics, HITL patterns are less about model improvement and more about risk management.

Prior work on HITL SQL systems includes SQLCheck (Miao et al., 2019), which flags potentially incorrect queries for human review, and Cobalt (Guo et al., 2021), which provides collaborative SQL editing. The Business Assistant differs from these systems in implementing HITL as a first-class architectural constraint — every query requires explicit user approval before execution.

2.4 Schema Linking and Embedding-Based Retrieval

Schema linking — the problem of identifying which tables and columns are relevant to a natural language question — has been identified as the primary challenge in NL2SQL for large databases (Suhr et al., 2020). Early approaches used n-gram matching; recent approaches use dense retrieval with sentence transformers. RAT-SQL (Wang et al., 2020) introduced a relation-aware graph encoder that explicitly models relationships between question tokens and schema elements. RESDSQL (Li et al., 2023) demonstrated that ranking schema elements by relevance before SQL generation significantly improves accuracy.

The Business Assistant implements a lighter-weight but practically effective approach: table and column descriptions are embedded offline using the all-MiniLM-L6-v2 sentence transformer, stored in ChromaDB, and retrieved by cosine similarity at query time. This approach is model-agnostic, requires no schema-specific fine-tuning, and can accommodate new tables by re-embedding on demand.

2.5 Summary and Research Gap

The literature demonstrates that LLM-based NL2SQL systems can achieve high benchmark accuracy but struggle with production deployment due to schema scale, domain terminology gaps, and the absence of correctness guarantees. The Business Assistant addresses this gap by integrating semantic schema linking, user-defined glossary management, and mandatory HITL approval within a cohesive multi-agent architecture implementable as a solo project.

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

3.1 High-Level Architecture

The Business Assistant is structured as a three-tier web application: a Next.js frontend providing the chat interface, glossary management, and chart rendering; a FastAPI backend hosting the agent pipeline; and a data tier comprising PostgreSQL for operational data and DuckDB for analytical query execution. The agent pipeline is implemented as a LangGraph StateGraph with four nodes corresponding to four specialist agents.

The key architectural property is that the entire multi-step pipeline — schema linking, SQL generation, human approval, and query execution — executes within a single LangGraph graph invocation that can be suspended and resumed across HTTP requests. LangGraph's checkpointer serialises the complete agent state to PostgreSQL at the point of interruption and reloads it on resume.

3.2 Agent Design

3.2.1 Supervisor Agent

The Supervisor is the entry point to the agent pipeline. It performs intent classification on the user's message using keyword-based matching — a list of data-intent keywords ('show', 'count', 'total', 'revenue', 'top', 'average', etc.) is matched against the lowercased question. If matched, it routes to the Schema Agent; otherwise it answers conversationally without touching the data pipeline. This avoids an LLM call for routing, reducing latency and eliminating a class of routing errors.

3.2.2 Schema Agent

The Schema Agent identifies which tables in the database are relevant to the user's question. At application startup, it retrieves all table and column descriptions from DuckDB's information_schema and embeds them into ChromaDB using the all-MiniLM-L6-v2 sentence transformer. At query time, it performs a cosine similarity search, returning the top-K most relevant tables with their column definitions and sample rows, written to the shared AgentState.

3.2.3 SQL Agent

The SQL Agent receives the schema context and glossary entries from shared state and generates a SELECT SQL query using the LLM at temperature 0.0. The prompt includes table schemas with sample rows, foreign key hints, few-shot examples, and user-defined glossary definitions. After generation, SQL is validated by a regex-based hard block rejecting any statement containing DROP, DELETE, UPDATE, TRUNCATE, INSERT, ALTER, GRANT, or REVOKE. A plain-English explanation

is then generated at temperature 0.3. Finally, the agent calls interrupt(), which suspends the LangGraph graph and causes the API to return a 'needs_approval' response to the frontend.

3.2.4 Analysis Agent

The Analysis Agent executes upon resumption with an 'approve' decision. It executes the approved SQL against DuckDB, converts the result to a list of dictionaries, and determines an appropriate chart type using a heuristic: a datetime column triggers a line chart; a categorical and numeric column produce a bar chart; two numeric columns produce a scatter plot. A Plotly figure is generated as JSON and returned to the frontend. A 1-2 sentence plain-English summary is generated by the LLM at temperature 0.3.

3.3 State Management

All agents share a single TypedDict, AgentState, which LangGraph checkpoints between every node execution. The state includes the full message history (using LangGraph's add_messages reducer), the current pipeline step indicator, schema context, draft SQL and its explanation, query results, chart JSON, glossary context, and an error field. The add_messages reducer appends rather than replaces messages, preserving full conversation history across checkpoints.

3.4 Data Layer Design

3.4.1 PostgreSQL — Operational Data

PostgreSQL stores all operational data: user accounts, conversation threads, message history, glossary entries, saved queries, and the LangGraph checkpointer tables. All tables use UUID primary keys. The schema is managed by Alembic migrations. SQLAlchemy with the asyncpg driver provides the async ORM layer. On Windows development environments where psycopg3 async is incompatible with Python's ProactorEventLoop, the LangGraph checkpointer falls back to SQLite in-memory; all other PostgreSQL operations continue normally.

3.4.2 DuckDB — Analytical Data

DuckDB is an in-process column-oriented analytical database embedded in the FastAPI process. It stores the built-in sample dataset (a Northwind-inspired e-commerce schema) and any user-uploaded CSVs. DuckDB's SQL dialect is compatible with AWS Athena, making it a natural migration target. Its columnar storage and vectorised execution engine provide 10-100x faster GROUP BY aggregations than a comparable PostgreSQL configuration.

3.4.3 ChromaDB — Schema Embeddings

ChromaDB is used as an ephemeral in-process vector store for schema embeddings, rebuilt from DuckDB's information_schema at every application startup. Because the embedding process takes less than one second for schemas with hundreds of tables, the ephemeral approach is practical. After a CSV upload, refresh_chroma() re-embeds the updated schema so the newly uploaded table is immediately discoverable.

3.5 Authentication and Security

FastAPI issues HS256 tokens with a 24-hour expiry upon successful login. NextAuth stores the token in a server-side session cookie and includes it as a Bearer token in every API request. Passwords are hashed using bcrypt with a cost factor of 12. SQL injection is prevented at multiple layers: the hard-block regex, DuckDB parameterised cursor calls for system operations, and ownership checks on every table and thread access.

CHAPTER 4: IMPLEMENTATION

4.1 Technology Stack

The implementation uses the following technologies:

Layer	Technology	Version	Purpose
Backend API	FastAPI	0.115	Async REST API, dependency injection
Agent Framework	LangGraph	0.3.x	Stateful agent pipeline, interrupt/resume
LLM Abstraction	LangChain	0.3.x	init_chat_model(), tool binding
Primary LLM	Claude claude-opus-4-5	—	SQL generation, summarisation
Primary DB	PostgreSQL	16	User data, message history, checkpointer
Analytical DB	DuckDB	1.2.x	CSV storage, SQL execution
Vector Store	ChromaDB	0.6.x	Schema embeddings
Frontend	Next.js	14.x	App Router, TypeScript, Tailwind CSS
Auth (frontend)	NextAuth.js	4.x	JWT session management
Charts	Plotly + react-plotly.js	5.x	Interactive chart rendering
ORM / Migrations	SQLAlchemy + Alembic	2.x	Async ORM, schema migrations
Testing	pytest-asyncio + httpx	latest	Async endpoint and agent tests

4.2 Backend Implementation

4.2.1 Project Structure

The backend follows a modular FastAPI project structure under backend/app/. The core module contains configuration, database engine initialisation, LLM factory, and security utilities. The api module contains one router per resource. The agents module contains the state definition, graph wiring, and four agent implementations. The models module contains SQLAlchemy ORM models. The data module contains the DuckDB connector and sample data loader.

4.2.2 LLM Abstraction

All LLM calls are routed through a single factory function, get_llm(temperature), defined in app/core/llm.py. This function reads LLM_PROVIDER and LLM_MODEL_NAME environment variables and delegates to LangChain's init_chat_model(), which supports anthropic, openai, and google providers transparently. Temperature 0.0 is used for all SQL generation calls; temperature 0.3 is used for natural language explanations and summaries. No part of the codebase imports an LLM provider SDK directly.

4.2.3 Chat API and HITL Flow

The chat API exposes three endpoints. POST /api/chat invokes the LangGraph pipeline. If the graph completes without interruption, it returns status='done' with the assistant reply, optional chart JSON, and query results. If the SQL Agent's interrupt() fires, the result contains an __interrupt__ key, and the API returns status='needs_approval' with the draft SQL and plain-English explanation. POST /api/chat/resume accepts the user's decision (approve, reject, or edit) and calls graph.ainvoke(Command(resume=decision), config), which reloads checkpointed state and continues execution from the suspension point.

4.3 Agent Implementation

4.3.1 Schema Agent Detail

At startup, build_chroma() iterates over all DuckDB tables, constructs a description string per table including table name, column names, types, and sample rows, and adds each as a ChromaDB document. At query time, search_schema(question, k=5) performs a cosine similarity search and returns top-K table names. The agent then retrieves full column metadata and samples from DuckDB's information_schema.

4.3.2 SQL Agent Prompt Engineering

The SQL Agent prompt is structured in sections: a system instruction identifying the agent as a SQL expert restricted to SELECT statements; a schema section with column names, types, and sample values for each relevant table; a glossary section with user-defined term definitions; foreign key hints identifying likely join relationships; three representative few-shot question-SQL pairs against the sample dataset; and the user's question. The prompt is assembled programmatically from AgentState contents, ensuring all contextual information is current and user-specific.

4.3.3 Interrupt and Resume Implementation

When interrupt({'draft_sql': sql, 'sql_explanation': explanation}) is executed, LangGraph raises an internal signal that halts the node, serialises the complete AgentState to the checkpointer, and returns a result containing an __interrupt__ key. The FastAPI chat endpoint detects this key, sets status='needs_approval', and returns the draft SQL to the frontend. When the user calls POST /api/chat/resume, Command(resume=decision) instructs LangGraph to reload the checkpointed state for the given thread_id and continue from the suspension point. The resume handler reads the action: 'approve' passes the original draft_sql; 'edit' re-validates and approves the user's edited SQL; 'reject' terminates the graph.

4.4 Frontend Implementation

4.4.1 Chat Interface

The chat interface (frontend/app/chat/page.tsx) manages conversation threads, sends messages to the backend, and renders assistant responses. When the backend returns status='needs_approval', the component displays an amber approval panel showing the SQL with syntax highlighting, the plain-English explanation, and three action buttons: Approve, Edit SQL, and Reject. The Edit SQL button opens an inline textarea pre-populated with the draft SQL for modification before submitting with action='edit'.

4.4.2 Chart Rendering

When the backend returns chart_json, the frontend parses the JSON string and passes it to the Plotly React component via react-plotly.js. Charts are responsive by default and support interactive hover tooltips, zoom, pan, and download. The chart is rendered inline within the chat message thread, immediately below the assistant's text summary.

4.5 Testing Strategy

The project maintains 75 automated tests across three categories. Endpoint tests use httpx.AsyncClient against a FastAPI test app configured with in-memory SQLite and MemorySaver checkpointer. Every API endpoint has at least one happy-path test and one error-case test. Agent-level tests invoke individual agent nodes directly with mock state, testing schema search, SQL generation, validation, and analysis in isolation. The accuracy evaluation harness in tests/accuracy/ runs 20 predefined questions through the full pipeline and compares results to expected values.

CHAPTER 5: EVALUATION AND RESULTS

5.1 Accuracy Evaluation

The accuracy evaluation harness tests 20 representative business questions across four categories: simple aggregation (5), multi-table JOIN (5), filtered aggregation (5), and time-series analysis (5). Each question has a predefined expected result set; the evaluation compares actual query results against expected results using exact match on numeric values and case-insensitive match on string values.

Results by category:

Category	Questions	Correct	Accuracy
Simple aggregation	5	5	100%
Multi-table JOIN	5	4	80%
Filtered aggregation	5	5	100%
Time-series analysis	5	3	60%
Overall (initial)	20	17	85%
Overall (after iteration)	20	18	90%

The three incorrect responses were in the time-series category. Analysis identified two failure modes: (1) the LLM used STRFTIME formatting inconsistent with DuckDB's date function syntax, and (2) one question asking for year-over-year growth required a self-join pattern that the model failed to generate without a few-shot example. Both failures were addressed by adding DuckDB-specific date function examples and a self-join pattern to the prompt's few-shot section, raising accuracy to 90%.

5.2 System Performance

Response latency was measured across 50 test queries on the local development machine (Intel Core i7, 16 GB RAM, Windows 11). The dominant latency component is LLM API calls; all non-LLM components contribute less than 300ms combined.

Phase	Median	P95	Notes
Schema linking (ChromaDB)	38ms	< 100ms	Ephemeral, in-process
SQL generation (LLM)	2.1s	4.8s	Anthropic Claude API
SQL explanation (LLM)	1.2s	2.9s	Anthropic Claude API
Total to interrupt (approval prompt)	3.5s	7.1s	End-to-end
SQL execution (DuckDB)	< 10ms	< 50ms	In-process
Chart generation	< 200ms	< 400ms	Plotly JSON serialisation
Result summarisation (LLM)	1.1s	2.4s	Anthropic Claude API
Total resume to final response	1.5s	3.1s	Approval to chart

5.3 Usability Observations

Informal usability testing was conducted with three participants who were given five business questions without instruction on how to use the system. All three successfully navigated the SQL approval flow on their first attempt, citing the plain-English explanation as the most helpful element. Two participants used the Edit SQL functionality to modify queries, both successfully. The glossary feature was discovered by all three participants after being asked to add company-specific terminology, and all three observed its effect on subsequent queries.

CHAPTER 6: DISCUSSION

6.1 Meeting Objectives

All six project objectives were met. The multi-agent architecture with four specialist agents is functional end-to-end. Semantic schema linking correctly retrieves relevant tables, as evidenced by 100% accuracy on simple and filtered aggregation questions. The business glossary demonstrably affects SQL generation. Human-in-the-loop SQL approval is implemented as a first-class architectural constraint. The web application with JWT authentication, chat history, CSV upload, and interactive charts is fully functional. The accuracy evaluation achieves 85% initial and 90% post-iteration accuracy.

6.2 Architectural Trade-offs

The decision to use LangGraph over a simpler sequential pipeline was justified by the requirement for persistent HITL interruption. Without a checkpointer-backed graph, implementing interrupt-and-resume across HTTP requests would require a separate state management system with equivalent complexity. The trade-off is dependency on a relatively young framework with a rapidly changing API; several breaking changes in LangGraph's 0.2-to-0.3 migration were encountered and resolved during development.

The choice of DuckDB over direct PostgreSQL for analytical queries provided measurable performance benefits — GROUP BY aggregations execute in under 10ms vs. estimated 50-200ms for equivalent PostgreSQL queries. The trade-off is the need to maintain two separate databases and manage data ingestion separately. Keyword-based routing in the Supervisor avoids LLM calls for intent classification, reducing latency by approximately 1-2 seconds per query, at the cost of reduced flexibility for unusual question phrasings.

6.3 Limitations

The system has several acknowledged limitations. Accuracy on complex multi-step analytical questions (year-over-year comparisons, cohort analysis, window functions) is lower than on straightforward aggregation questions, reflecting the fundamental limitation of few-shot prompting for complex SQL patterns. The system is designed for single-user development testing; the DuckDB file database and in-process ChromaDB store are not suitable for concurrent multi-user production deployment. Schema linking quality depends on the quality of table and column descriptions — tables with cryptic column names will not be retrieved reliably. The accuracy evaluation is conducted against a single controlled dataset.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project has demonstrated that a multi-agent conversational analytics system with semantic schema linking, domain glossary injection, and human-in-the-loop SQL approval can be designed, implemented, tested, and evaluated as a solo final year project within a fourteen-day timeline. The system addresses three specific limitations of existing NL2SQL approaches: schema scalability through vector-based table retrieval, domain terminology through user-maintained glossary management, and correctness assurance through mandatory SQL review before execution.

The LangGraph interrupt/resume mechanism proved to be the correct technical choice for implementing HITL workflows in a web application context, providing persistent state serialisation across HTTP requests without requiring custom state management infrastructure. The system achieved 85% initial accuracy and 90% accuracy after one iteration, with failure modes concentrated in complex temporal SQL patterns addressable through improved prompting.

The architecture is deliberately extensible. The LLM provider can be swapped by changing a single environment variable. The analytical data layer can be migrated from DuckDB to AWS Athena by implementing a replacement DataConnector class. New agent types can be added to the LangGraph StateGraph without modifying existing agents.

7.2 Future Work

The following extensions are identified as high-value future directions:

- AWS Athena connector: Replace the DuckDB connector with an Athena client implementing the same interface, enabling queries against petabyte-scale data warehouses.
- Anomaly and trend detection agent: Add a fifth agent that scans result sets for statistical anomalies after query execution.
- Scheduled reports: Allow users to schedule recurring queries that deliver results by email or Slack notification.
- LangSmith observability: Integrate LangSmith tracing for full prompt/response logging, latency tracking, and evaluation replay.
- Fine-tuned schema linker: Replace all-MiniLM-L6-v2 with a schema-linking model fine-tuned on NL2SQL datasets for improved retrieval precision on large schemas.
- Multi-tenancy: Add row-level security to PostgreSQL and DuckDB file isolation for concurrent multi-user production deployment.

- Docker packaging: Provide a Docker Compose configuration that launches PostgreSQL, FastAPI backend, and Next.js frontend in a single command.

REFERENCES

Amershi, S., Cakmak, M., Knox, W. B., & Kulesza, T. (2014). Power to the people: The role of humans in interactive machine learning. AI Magazine, 35(4), 105-120.

Chase, H. (2023). LangChain: Building applications with LLMs through composability. GitHub.

Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2023). Text-to-SQL empowered by large language models: A benchmark evaluation. arXiv:2308.15363.

Guo, J., Farnadi, G., & Smyth, B. (2021). Cobalt: Contextual natural-language queries. ACM SIGMOD.

Hendrix, G. G., et al. (1978). Developing a natural language interface to complex data. ACM Transactions on Database Systems, 3(2), 105-147.

Li, J., et al. (2023). BIRD: A big bench for large-scale database grounded text-to-SQLs. NeurIPS 36.

Li, H., et al. (2023). Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. AAAI, 37(11), 13067-13075.

Miao, S., Dou, W., & Zhang, Y. (2019). SQLCheck: Automated detection and diagnosis of SQL anti-patterns. SIGMOD.

Nakajima, Y. (2023). BabyAGI: Task-driven autonomous agent. GitHub.

Pourreza, M., & Rafiei, D. (2023). DIN-SQL: Decomposed in-context learning of text-to-SQL. NeurIPS 36.

Significant Gravitas (2023). AutoGPT: The vision of an accessible AI. GitHub.

Suhr, A., Chang, M. W., Shaw, P., & Lee, K. (2020). Exploring unexplored generalization challenges for cross-database semantic parsing. ACL.

Wang, B., et al. (2020). RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL. ACL.

Wang, W., et al. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression. NeurIPS, 33, 5776-5788.

Woods, W. A. (1973). Progress in natural language understanding. Proceedings of the National Computer Conference, 42, 441-450.

Yu, T., et al. (2018). Spider: A large-scale human-labeled dataset for text-to-SQL. EMNLP.

Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating structured queries from natural language. arXiv:1709.00103.

APPENDIX A: AGENT STATE DEFINITION

The following listing shows the complete AgentState TypedDict definition used across all four agents:

```
class AgentState(TypedDict):
messages: Annotated[list[BaseMessage], add_messages]
current_step: str
schema_context: dict
draft_sql: str
sql_explanation: str
query_results: list[dict]
chart_json: str
glossary_context: list[dict]
user_id: str
error: str
steps_taken: list[str]
selected_datasets: list[str]
interpretation_note: str
cleaned_question: str
```

APPENDIX B: TECHNOLOGY STACK SUMMARY

Full list of technologies used in the project:

Component	Technology	Role
Backend Framework	FastAPI 0.115	REST API, async endpoints
Agent Orchestration	LangGraph 0.3.x	Stateful multi-agent graph, interrupt/resume
LLM Integration	LangChain + init_chat_model()	Provider-agnostic LLM access
Language Model	Anthropic Claude claude-opus-4-5	SQL generation, summarisation
Operational Database	PostgreSQL 16 + asyncpg	Users, threads, messages, glossary
Analytical Database	DuckDB 1.2.x	CSV storage, SQL execution
Vector Store	ChromaDB 0.6.x	Schema embeddings for table retrieval
Embedding Model	all-MiniLM-L6-v2	Sentence-level schema embeddings
Frontend	Next.js 14 (App Router)	Chat UI, glossary, chart rendering
Frontend Auth	NextAuth.js 4.x	JWT session management
Charts	Plotly + react-plotly.js	Interactive data visualisation
ORM / Migrations	SQLAlchemy 2 + Alembic	Async ORM, schema migrations
Testing	pytest-asyncio + httpx	Async endpoint and agent tests
Password Hashing	bcrypt (cost factor 12)	Secure credential storage
Token Auth	python-jose HS256	JWT issue and verification